

ECON1059 Seminar 1

Notes for the Seminar Series on Python Programming for Economics (ECON1059, Fall 2025)

Juncheng ZHANG, PhD Candidate in Economics, UNNC.

Research interest: Behavioral and Experimental Economics.

Email: Juncheng.ZHANG@nottingham.edu.cn

Hello :D & Welcome :)

About Python

- Python is friendly with a low entry barrier (we moved it to Year 2 after all :))
- It's powerful for data analysis and simulation (No worry—we'll keep it basic).
- New to Python? Fantastic :D This is your chance to learn it properly, which will be very useful later on.

About the Seminar Series

- Seminar runs on a weekly basis (longer than most Moodle).
- The BEST way to learn any language—coding or speaking—is just by *doing*.
- We're here to help you go through the **problem sets**.
 - We strongly recommend trying the problem sets *before the seminar*—this will make our sessions more efficient and your experience much smoother.
 - We won't cover every question (time is short, and you already have solutions-fair).
 - We're here to help (of course!). But bug-hunting is tricky for humans :P. Ask an AI bot first—if it's still confusing, email me.

Outline

- Install Python
- `type()`
- `print()`
- functions

Install Python

- University computer: Anaconda is already installed, so you can directly open and use it.
- Your laptop(desktop):
 - Option 1: Setup Python and then use terminal to install Jupyter Notebook.
 - Option 2 (Recommended): Setup [Anaconda](#): create an environment, then install Jupyter Notebook and use it in [Visual Studio Code](#) (this is the way I use).

Data Type

- Numbers:
 - Integers, `int` . such as `0,1,2`
 - Float, `float` . such as `1.1, 1.0`
- String `str` (quotation mark " " or ' ')
 - Number string. Such as `"1", "2"`
 - Text string. Such as `"Some text"`

To get the data type by using `type()` function and use `print()` function to show the results.

```
In [1]: print("hello world")
```

```
hello world
```

```
In [2]: print(type("hello world"))
```

```
<class 'str'>
```

```
In [3]: print("""this is
a multi-line
string.
""")
```

```
this is
a multi-line
string.
```

- What happens if we run the following line of code `print(hello)` ? Why?

```
In [4]: print(hello)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[4], line 1
----> 1 print(hello)

NameError: name 'hello' is not defined
```

```
In [5]: # variable_name = value
hello = 1
print(hello)
```

```
1
```

```
In [6]: print(type(hello))
```

```
<class 'int'>
```

```
In [7]: # float
x = 2.0
print(type(x))
```

```
<class 'float'>
```

```
In [8]: # string
y = "hello"
print(type(y))
```

<class 'str'>

```
In [9]: # true division is always float
print(type(6/3))
```

<class 'float'>

```
In [10]: 6//4
```

Out[10]: 1

Functions

```
In [11]: ## Example 1
def function_name1(parameter1, parameter2):
    add = parameter1 + parameter2
    return add
```

```
In [12]: ## Example 1
a = 1
b = 3

results_1 = function_name1(a, b)
print(results_1)
```

4

```
In [13]: # formatted string
print(f"The add results is {results_1}")
```

The add results is 4

```
In [14]: print("The add results is", results_1)
```

The add results is 4

```
In [15]: ## Example 2  
def function_name2(parameter3, parameter4):  
    print("Hello")  
    return None
```

```
In [16]: ## Example 2  
result_2 = function_name2(a,b)
```

Hello

```
In [17]: ## Example 3  
def function_name3(parameter5, parameter6):  
    print("Hello")  
    return "World"
```

```
In [18]: ## Example 3  
result_3 = function_name3(a,b)
```

Hello

```
In [19]: print(result_3)
```

World

- Write a function that (a) multiplies 3 numbers with each other, (b) calculates $(x + y)^2$, and (c) solves some other mathematical problem of your liking (e.g., the length of the hypotenuse in a right triangle?).

```
In [20]: def multiply_three_numbers(x, y, z):  
    return x*y*z
```

```
In [21]: def quadratic_of_a_sum(x, y):  
    return (x + y)**2
```

```
In [22]: def hypotenuse(a, b):  
    return (a**2 + b**2)**0.5
```

- Next, consider the problem from the lecture, where we calculated $1 + 2 + 3 + 4 + 5 + 6 + 7 + 8$. Write a general function that solves this problem for $n + (n + 1) + (n + 2) + \dots + (n + k)$.

```
In [23]: # Arithmetic sequence
def sum_sequence(x_min, x_max):
    result = (x_min+x_max) * (x_max-x_min+1)/2
    return result
```

```
In [24]: print(sum_sequence(1,10))
```

55.0

- How about we combine that silly idea of multiplying text to draw boxes? (We will cover much better solutions to this soon)

```
In [25]: def draw_rectangle(width):
    print(width*"#")
    print(width*"#")
    print(width*"#")
```

```
In [26]: draw_rectangle(2)
```

```
##
##
##
```

```
In [27]: draw_rectangle(8)
```

```
#####
#####
#####
```